

Project 1 Report Notes

姓名：于翔 学号：23307130202

2026 年 4 月 26 日

1 MLP Baseline

本节记录 Part A 的实现与实验结果。按照项目要求，本部分只完成 MLP baseline，不引入 CNN、momentum、learning rate scheduler、L2 regularization、dropout 等后续比较项，避免基线设置与 Part B/Part C 混淆。

1.1 Implementation

本部分主要修改了以下文件：

- `codes/mynn/op.py`: 实现线性层前向、反向传播和 softmax cross entropy 损失。
- `codes/mynn/models.py`: 修正 `Model_MLP` 的初始化与模型加载逻辑，并使用两层隐藏层以便和后续小型 CNN 的模型规模更接近。
- `codes/mynn/optimizer.py`: 修正 `SGD.step`，使用原地参数更新，确保 `Linear.forward` 中实际使用的 `W` 和 `b` 被更新。
- `codes/mynn/runner.py`: 修正训练循环中的空 batch 问题，并将验证改为按固定间隔执行。
- `codes/test_train.py`: 设置并运行 MLP baseline 训练流程。
- `codes/test_model.py`: 加载保存的 baseline 模型并在测试集上评估。

线性层前向传播为

$$Y = XW + b.$$

反向传播中，设下一层传回梯度为 $\frac{\partial L}{\partial Y}$ ，则

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial b} = \sum_i \frac{\partial L}{\partial Y_i}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

分类损失采用 softmax cross entropy。实现中先对 logits 做数值稳定的 softmax，再计算平均交叉熵损失。反向传播使用 softmax 与 cross entropy 合并后的梯度：

$$\frac{\partial L}{\partial z} = \frac{p - y}{N},$$

其中 p 为 softmax 概率， y 为 one-hot 标签， N 为 batch size。

1.2 Training Setting

MNIST 训练集共 60000 张图像。实验中固定随机种子为 309，将其中 10000 张作为 validation set，其余 50000 张作为 training set。输入图像展平为 784 维向量，并归一化到 $[0, 1]$ 。

Item	Setting
Model	MLP
Architecture	$784 \rightarrow 32 \rightarrow 16 \rightarrow 10$
Activation	ReLU
Loss	Softmax cross entropy
Optimizer	SGD
Learning rate	0.1
Batch size	128
Epochs	5
Validation size	10000

表 1: Part A MLP baseline setting.

1.3 Results

训练完成后，保存的最佳模型位于 `codes/best_models/mlp/best_model.pickle`。学习曲线保存于 `codes/figs/part_a_mlp_learning_curve.png`。

Metric	Accuracy
Best validation accuracy	0.94790
Test accuracy	0.95440

表 2: Part A MLP baseline results.

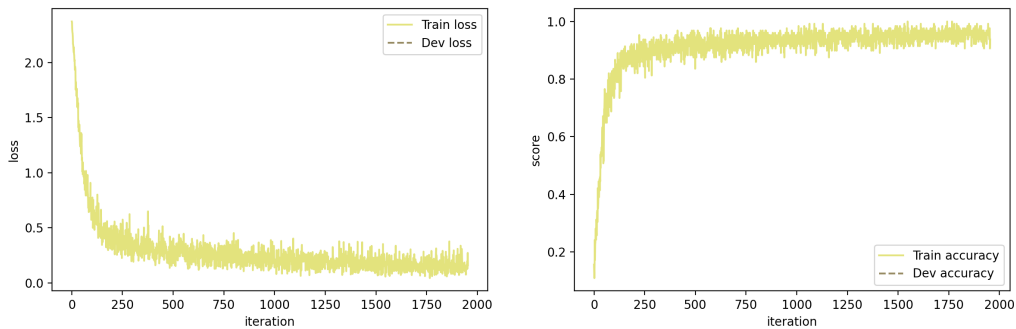


图 1: Learning curve of the MLP baseline.

2 CNN Model and MLP-vs-CNN Comparison

本节记录 Part B 的实现与实验结果。按照项目要求，本部分手动实现 `conv2D`，并基于该算子构建一个简单 CNN，在相近训练设置下与 Part A 的 MLP baseline 进行比较。

2.1 Implementation

本部分主要修改或新增了以下文件：

- `codes/mynn/op.py`: 实现 `conv2D.forward` 和 `conv2D.backward`，并新增 Flatten 与 MaxPool2D 层。
- `codes/mynn/models.py`: 实现 `Model_CNN`，包括前向传播、反向传播、模型保存与加载。
- `codes/mynn/runner.py`: 支持按 batch 分批验证，避免 CNN 在验证集上一次性缓存过大的 feature map。
- `codes/test_train.py`: 通过参数 `--model cnn` 训练 CNN 模型并保存学习曲线。
- `codes/test_model.py`: 通过参数 `--model cnn` 加载保存的 CNN 模型并在测试集上评估。

`conv2D` 的输入为 $[N, C, H, W]$ ，卷积核为 $[O, C, K, K]$ 。实现中使用 NumPy 的滑动窗口视图构造局部感受野，并使用 `einsum` 完成卷积计算；反向传播中手动计算卷积核梯度、bias 梯度和输入梯度。该实现没有调用任何深度学习框架或高级神经网络算子。

2.2 CNN Training Setting

CNN 采用一个较小的两层卷积结构，便于在 CPU 上完整训练、复现实验，并为后续卷积特征可视化保留更丰富的中间表示：

Conv($1 \rightarrow 4, 3 \times 3, \text{padding} = 1$) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
 $\rightarrow$ Conv($4 \rightarrow 8, 3 \times 3, \text{padding} = 1$) $\rightarrow$ ReLU $\rightarrow$ Flatten $\rightarrow$ Linear($8 \times 14 \times 14 \rightarrow 10$).

训练集、验证集划分、随机种子、batch size 和 epoch 数均与 MLP baseline 保持一致。CNN 使用固定学习率 SGD，不使用 momentum、learning rate scheduler 或额外正则化。第一层卷积后加入 2×2 max pooling，使第二层卷积在 14×14 特征图上运行；这样仍然保留两层卷积结构，同时降低手写 NumPy 卷积的训练成本。

2.3 Comparison Results

CNN 训练完成后，保存的最佳模型位于 `codes/best_models/cnn/best_model.pickle`。学习曲线保存于 `codes/figs/part_b_cnn_learning_curve.png`。

Item	Setting
Model	CNN
Architecture	Conv-ReLU-Pool-Conv-ReLU-Flatten-Linear
Conv channels	4, 8
Kernel size	3×3
Padding	1
Pooling	2×2 max pooling
Loss	Softmax cross entropy
Optimizer	SGD
Learning rate	0.05
Batch size	128
Epochs	5
Validation size	10000

表 3: Part B CNN setting.

Model	Best validation accuracy	Test accuracy
MLP baseline	0.94790	0.95440
CNN	0.97570	0.97790

表 4: MLP-vs-CNN comparison under similar training settings.

2.4 Discussion

在当前设置下, CNN 的 validation accuracy 和 test accuracy 都高于 MLP baseline。主要原因是 CNN 的卷积核共享参数, 并利用了图像中的局部空间结构; 相比直接把 28×28 图像展平成向量的 MLP, CNN 更容易学习局部笔画、边缘和小形状等与手写数字相关的模式。

本实验中的 CNN 仍然很简单, 只包含两层小宽度卷积和一个 max pooling 层, 没有使用 dropout、batch normalization 或复杂残差结构。相较于单卷积层版本, 两层卷积提供了更适合后续分析和可视化的中间特征; 相较于更宽的 $8 \rightarrow 16$ 版本, $4 \rightarrow 8$ 的通道设置更适合当前手写 NumPy 实现的运行成本。

2.5 Current Verification

已完成以下检查:

- `python3 -m py_compile` 检查通过。
- `python3 test_train.py --model mlp` 可完成 MLP 训练并保存模型与更宽的学习曲线图。
- `python3 test_model.py --model mlp` 可加载保存模型并输出测试准确率 0.9544。

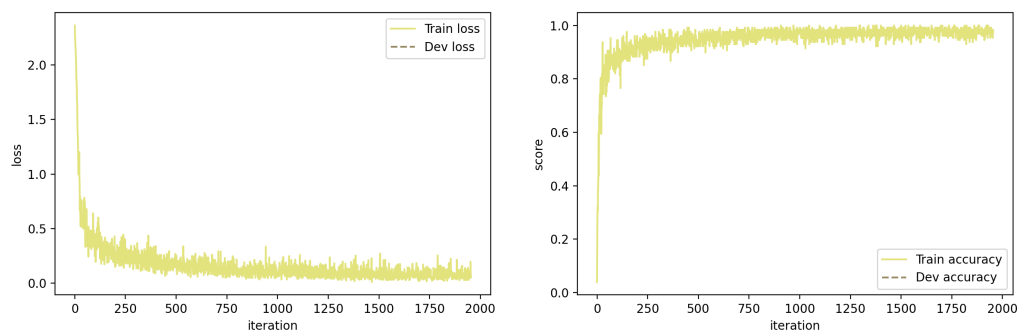


图 2: Learning curve of the CNN model.

- `python3 test_train.py --model cnn` 可完成两层 CNN 训练并保存模型与更宽的学习曲线图。
- `python3 test_model.py --model cnn` 可加载保存 CNN 模型并输出测试准确率 0.9779。
- 对 `conv2D.backward` 做了小规模数值梯度检查，卷积核梯度相对误差约为 1.19×10^{-13} 。